SNN: Final hypotheses report (25th September - 12th Oct)

Omar Abbas Haque

October 2026

1 Initial story

It is hard to explain (and understand) why I came up with those hypotheses unless you already know the story I had in mind while making them, so I'm writing this section at the start:
The time of the spike coincides with when the number of untrained neurons hits 0. Untrained neurons have an extremely low threshold; when some of them win the initial votes from the 350 ms presentations, they become imprinted with digit-resembling synaptic weights, but still with a low threshold (call them hijackers), so we can assume they're hijacking the votes and keeping the accuracy down (this is the cause for our previously seen pre-spike biases). Once all of them have been trained to a threshold high enough, accuracy suddenly rises because there are no hijackers.
The spike hence, only happens because the number of neurons in this "reservoir" of untrained neurons hits 0.
Detailed mechanistic conclusion with all moving pieces at the end of this PDF.

2 Terminology

I will introduce some of these as the hypotheses progress, but I'm putting all of them here as well for quick recall.

1. Held-out accuracy: Accuracy estimation while training, measured by evaluating on a subset of 3000 images from MNIST. I had to give this a name because I tried incorporting your method of accuracy estimation (the one you used in your write-up) as well.
2. Spike: The spike in accuracy, measured as the first snapshot (every 100 batches, or 3200 data points) after which accuracy never drops below 78 % again.
3. Reservoir: Excitatory neurons whose template is still approximately equal to its random initialization. In the code, I calculate this as the number of excitatory neurons with less than 35 % of their weight on their top-100 pixels. For reference, a uniform (0,01, 0,3) scores 23 %, and formed digit templates score 50-80%.
4. Recruitable neurons: Neurons in the reservoir with $\theta \approx -52.0$ mV (the default starting value)
5. Trained neurons: Neurons in the reservoir have barely spiked and have near-initialization threhsolds and synapses. Once they spike, they get converted to "trained neurons", with a clear imprint of a digit and higher spiking thresholds.
6. Window: The interval between two consecutive snapshots, i.e. 100 batches = 3200 training examples. All per-window quantities in my figures are totals over every 350 ms poisson spike presentation in that interval.
7. θ: Adaptive threshold of the excitatory neurons
8. Neurons: Excitatory neurons

3 Hypotheses

1. Hypothesis 1 (baseline): The number of excitatory neurons in the reservoir decays linearly and reaches 0 at the time of the spike.
2. Hypothesis 2 (refill): If, at a certain point in training, we intervene by refilling the reservoir, and by resetting n trained neurons to their initialization states, the accuracy spike will get delayed by a factor linearly proportional to n.

3. Hypothesis 3 (pseudo-refill): If, at a certain point in training, we intervene by resetting n trained neurons to their initialization states, but make their adaptive thresholds equal to the median of the trained neuron populations' threshold, the spike will still happen at the same time, but it will have a lower peak accuracy.

4. Hypothesis 4 (sabotage): If, at a certain point in training, we intervene by raising the threshold of every neuron in the reservoir to the median of the trained neurons, but don't change their weights, the spike will happen near-instantaneously (within 5-10k examples), but it will have a lower peak accuracy (depending on how early we sabotage, i.e how many reservoir neurons we sabotage)

5. Hypothesis 5 (imprint): If, at a certain point in training, we intervene by giving each reservoir neuron a random training image as template (normalized to 78.4), and set their θ = trained median, the spike happens near instantaneously (within 5-10k examples)

6. Hypothesis 6 (hijacking): The neurons that are "recruited" (moved from the recruitable reservoir to trained neurons) dominate the network for a few data points, before their threshold reaches high enough for another reservoir neuron to win and repeat the process. Hijacking stops at the accuracy spike

4 Hypothesis testing

4.1 Hypothesis 1 (baseline): The number of excitatory neurons in the reservoir decays linearly and reaches 0 at the time of the spike.

Accepted.

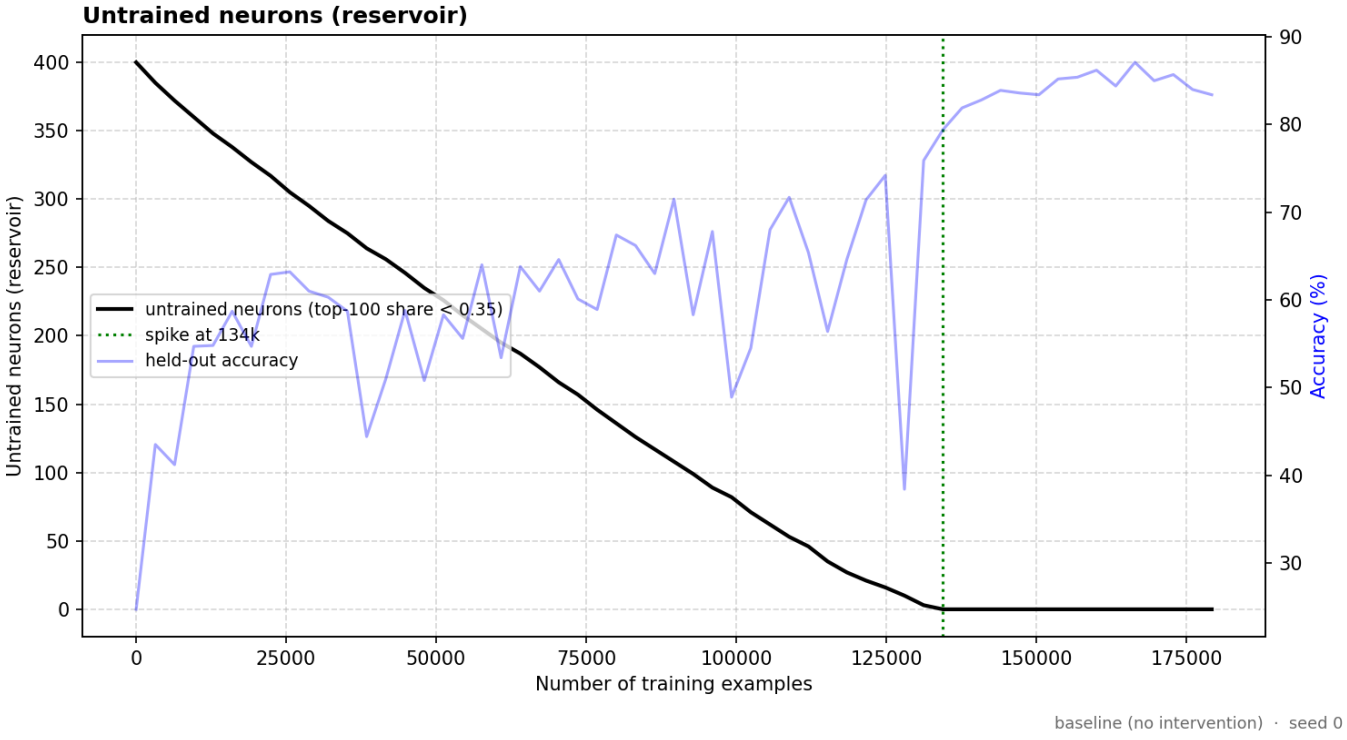
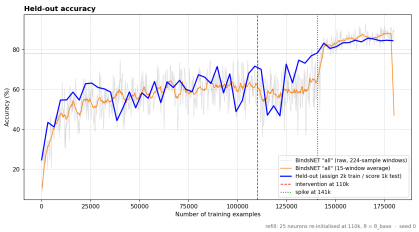


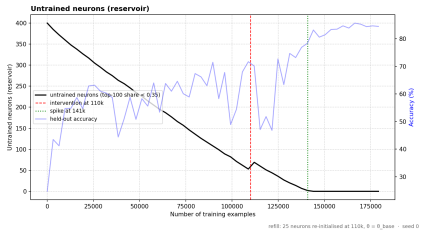
Figure 1: Number of neurons in the reservoir decays linearly

4.2 Hypothesis 2 (refill: If, at a certain point in training, we intervene by refilling the reservoir, and by resetting n trained neurons to their initialization states, the accuracy spike will get delayed by a factor linearly proportional to n.)

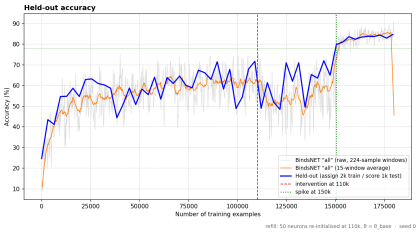
This hypothesis refills the reservoir and leaves the neurons in it recruitable.
Accepted. This seed has the spike at 134k training examples without intervention (fig. 1). Spike is at 141k for n=25 refilled neurons (fig. 2(a)), 150k for n=50 (fig. 2(c)), and 166k for n=100 (fig. 2(e)). This is linear (with some noise).



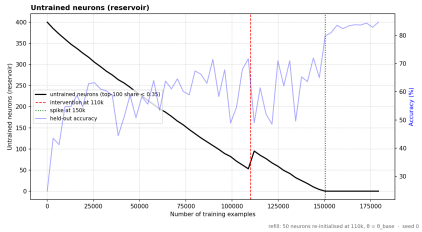
(a) Refill n=25 trained neurons: accuracy



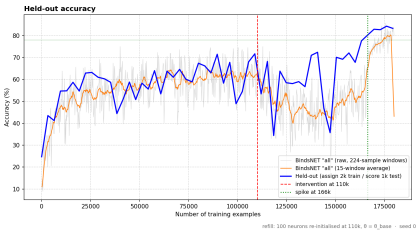
(b) Refill n=25 trained neurons: reservoir



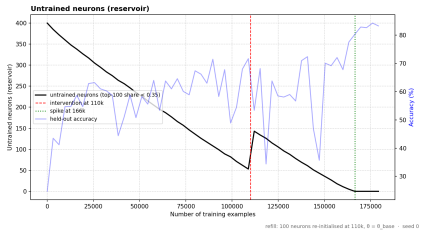
(c) Refill n=50 trained neurons: accuracy



(d) Refill n=50 trained neurons: reservoir



(e) Refill n=100 neurons: accuracy

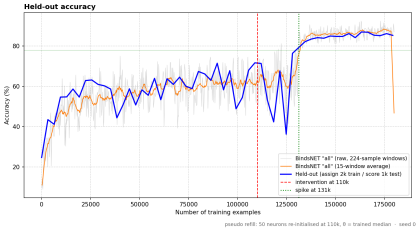


(f) Refill n=100 neurons: reservoir

Figure 2: Refill at different n values, seed 0

4.3 Hypothesis 3 (pseudo-refill): If, at a certain point in training, we intervene by resetting n trained neurons to their initialization states, but make their adaptive thresholds equal to the median of the trained neuron populations' threshold, the spike will still happen at the same time, but it will have a lower peak accuracy.

This hypothesis refills the reservoir but leaves the neurons in it unrecruitable.
Accepted (partially), the peak accuracy is not lower. Maybe we need to pseudo-refill more than 50 neurons to see a change in accuracy. This counters the argument that if refilling was somehow "breaking" the network, and the delay in spike is the network "recovering", then in this case, we should've seen a delayed spike as well. However, that is not the case, proving that the only thing that matters when it comes to the spike is the presence of recruitable reservoir neurons.
It is also interesting to note that fig.3(b) ends at 50 reservoir/untrained neurons, which means that once neurons join the theta levels for the trained group, they are never able to train again.



(a) Pseudo refill n=50 reservoir neurons: accuracy

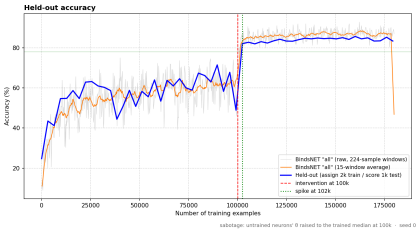


(b) Pseudo refill n=50 reservoir neurons: reservoir

Figure 3: Pseudo-refill n=50 reservoir neurons 110k training points, seed 0

4.4 Hypothesis 4 (sabotage): If, at a certain point in training, we intervene by raising the threshold of every neuron in the reservoir to the median of the trained neurons, but don't change their weights, the spike will happen near-instantaneously (within 5-10k examples), but it will have a lower peak accuracy (depending on how early we sabotage, i.e how many reservoir neurons we sabotage)

This hypothesis converts all recruitable reservoir neurons to unrecruitable reservoir neurons.
Accepted (partially). I only tested on sabotage at 100k training points, and accuracy was not markedly lower (I am very confident sabotaging earlier will lower our accuracy, though).
As seen in hypothesis 3 as well, fig.4(b) ends at a non-zero number of reservoir/untrained neurons, again reinforcing the fact that once neurons join the theta levels for the trained group, they are never able to train again.



(a) Sabotage all reservoir neurons at 100k training examples: accuracy

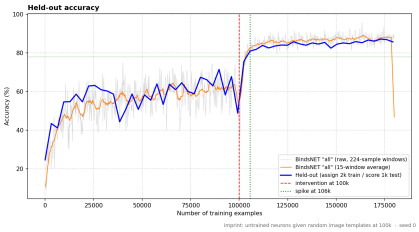


(b) Sabotage all reservoir neurons at 100k training examples: reservoir

Figure 4: Sabotage at 100k training examples, seed 0

4.5 Hypothesis 5 (imprint): If, at a certain point in training, we intervene by giving each reservoir neuron a random training image as template (normalized to 78.4), and set their θ = trained median, the spike happens near instantaneously (within 5-10k examples)

This hypothesis empties the reservoir entirely and doesn't any neurons recruitable after the intervention.
Accepted.



(a) Imprint all reservoir neurons at 100k training examples: accuracy



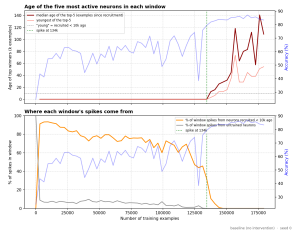
(b) Imprint all reservoir neurons at 100k training examples: reservoir

Figure 5: Imprint at 100k training examples, seed 0

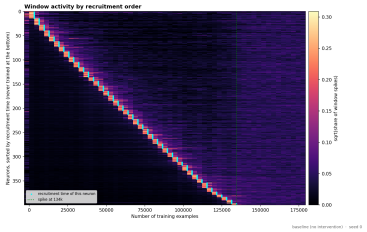
4.6 Hypothesis 6 (hijacking): The neurons that are "recruited" (moved from the recruitable reservoir to trained neurons) dominate the network for a few data points, before their threshold reaches high enough for another reservoir neuron to win and repeat the process. Hijacking stops at the accuracy spike

Accepted.

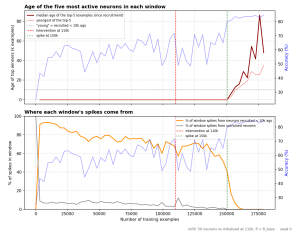
How to read left side figures (two-panels). Every 3200 training examples the run takes a snapshot. For each snapshot window, the **top panel** picks the five neurons that fired the most and plots how long ago each of them was recruited (dark red: median of the five, salmon: the youngest of the five). The **bottom panel** answers the question



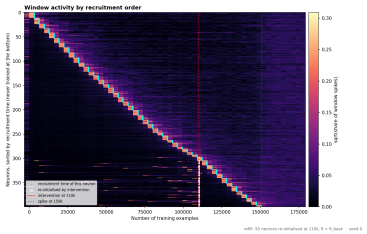
(a) Baseline: Winner age



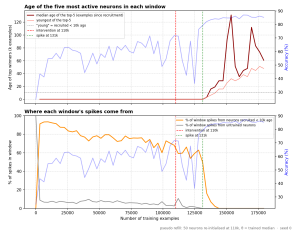
(b) Baseline: Window activity



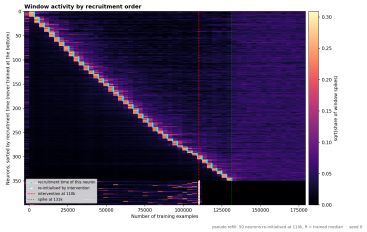
(c) Refill(n=50): Winner age



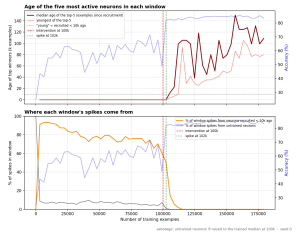
(d) Refill(n=50): Window activity



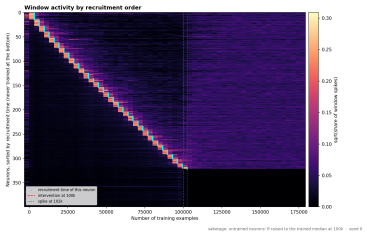
(e) Pseudo refill(n=50): Winner age



(f) Pseudo refill(n=50): Window activity



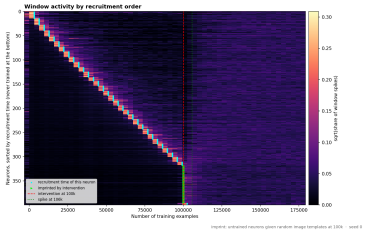
(g) Sabotage: Winner age



(h) Sabotage: Window activity



(i) Imprint: Winner age



(j) Imprint: Window activity

Figure 6: Age of winner (left), and activity at recruitment (right)

of where the window's spikes came from: the orange line is the percentage of all output spikes produced by neurons recruited less than 10k examples ago, the grey line is the percentage produced by neurons that are still untrained.

Observations: during the plateau the salmon line is flat at zero and the orange line sits at 70–90 %. i.e, in every window the neurons winning the votes were recruited *in that same window*. Recent recruits are running the network, and a different new recruit does so in the next window. At the spike both lines change at once: nobody new is recruited any more, so the age of the winners simply grows with time, and the orange line drops to zero within 10k examples (trivial, since no neuron is young any more).

How to read right side figures (the colorful ones) One row per neuron (all 400), one column per snapshot window. Rows are sorted by when the neuron was recruited, i.e. when its synapses first became digit-like: first recruit at the top, neurons that never got trained at the bottom. Colour is the neuron's share of that window's spikes. A cyan dot marks the neuron's recruitment time; white < marks a neuron re-initialised by the intervention, lime > a neuron imprinted by it.

Observations:

- The cyan bars form a straight diagonal, i.e recruitment happens one neuron at a time at a steady rate (hypothesis 1).
- Immediately around each cyan bar there is a bright color that fades within a few windows. That color is the hijacking: a newly recruited neuron takes a large share of all spikes for roughly 10k examples, until its θ has risen enough for the next reservoir neuron to out-compete it on the 350 ms presentations. The bright color streak starts in the column just left of the bar because the bar marks the *end* of the window in which the synapses became digit-like – the neuron was already winning during that window.
- The band ends at the green line. Right of the green line the picture is uniform; spikes are spread over all 400 rows and no neuron dominates.

Differences between baseline, refill, pseudo-refill, sabotage, and imprint

- **Sabotage and imprint at 100k:** No cyan bars to the right of the red line (point of intervention) for both. This is because both interventions leave no recruitable neurons, which are the neurons that dominate their respective windows. Sabotaged neurons have initialization weights, so they can't win the spike race against the actually trained neurons (hence the (h) figure is entirely black after the red line for those neurons). Imprinted neurons (j) can still spike because they have good templates for their synapses.
- **Refill($n = 50$) at 110k:** White < markers at 110k on the 50 reset neurons (they are re-sorted by their second recruitment, so they sit below the original recruits). To the right of the red line they acquire cyan bars one by one, and the staircase starts again.
- **Pseudo-refill($n = 50$) at 110k:** No cyan bars ever follow after the intervention. This happens because a pseudo-refill renders all neurons in the reservoir unrecruitable. The 50 rows sit at the very bottom. A reset template alone does not make a hijacker; the low θ does.

5 Overall conclusion

Some terminology (pasted from the terminology section on the first page):
Reservoir: Excitatory neurons whose template is still approximately equal to its random initialization. In the code, I calculate this as the number of excitatory neurons with less than 35 % of their weight on their top-100 pixels. For reference, a uniform (0,01, 0,3) scores 23 %, and formed digit templates score 50-80%.
Recruitable neurons: Neurons in the reservoir with $\theta \approx -52.0$ mV (the default starting value)

The spike happens because neurons with strong training data imprints, but low thresholds (hijackers) stop hijacking the votes. By "stop", I mean the network runs out of potential hijackers:
At the start of training, few random neurons with favorable initializations dominate the network. They learn quickly through STDP and get a strong training data imprint on their synapses, which causes them to spike further. A hijacker's votes are wrong because a hijacker wins images of several classes but can carry only one label. Eventually, their thresholds get too high too high to beat a hijacker to the first spike. At this point, the neurons get separated into two classes:

1. the recruitable "reservoir", i.e the neurons with near-initialization synaptic weights and near-initialization thresholds,

2. the trained neurons, with strong imprints but thresholds too high to win the firing race.

Continuing, the few random neurons in the recruitable reservoir with favorable initializations begin to hijack the network (they get "recruited"), only to similarly push their thresholds into trained neuron territory.
This process repeats until the recruitable reservoir is empty. If the recruitable reservoir is empty, there can be no more hijackers, hence the trained neurons take over and push the accuracy up.
My testing showed that the supply of recruitable neurons is what sets the time of the spike. You can control (and exactly calculate) when the spike happens if you know the number of neurons in the recruitable reservoir. If you empty the recruitable reservoir forcibly mid-training: accuracy spikes. Refill it: the spike is delayed in proportion; Refill it with high-θ neurons (not recruitable): nothing changes